САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 6

Мониторинг
микросервисов

Выполнил:

Губанов Егор

БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Настроить мониторинг микросервисного приложения с помощью Grafana, Prometheus, Loki и Promtail. На дашборде показать CPU, RAM, диск и сеть виртуальной машины, доступность всех микросервисов и логи контейнеров. Дополнительно настроить почтовое уведомление об изменении доступности и интегрировать self-hosted Sentry для сбора ошибок приложения.

Ход работы

1 Проверка приложения и ресурсов

Подключился к Debian по SSH и проверил память, диск и контейнеры приложения из ЛР5. Все десять контейнеров работали со статусом healthy. Для мониторинга увеличил память виртуальной машины до 4 ГБ.

```
vboxuser@foransible:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       1.1Gi       109Mi        94Mi       962Mi       889Mi
Swap:          550Mi        12Ki       550Mi
vboxuser@foransible:~$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3       8.6G  3.0G  5.2G  37% /
vboxuser@foransible:~$ docker ps --format "table {{.Names}}\t{{.Status}}"
permission denied while trying to connect to the docker API at unix:///var/run/d
ocker.sock
vboxuser@foransible:~$ sudo docker ps --format "table {{.Names}}\t{{.Status}}"
[sudo] password for vboxuser:
NAMES                     STATUS
rent-lab5-gateway-1       Up 37 minutes (healthy)
rent-lab5-messaging-1     Up 37 minutes (healthy)
rent-lab5-deals-1         Up 37 minutes (healthy)
rent-lab5-catalog-1       Up 37 minutes (healthy)
rent-lab5-auth-1          Up 37 minutes (healthy)
rent-lab5-rabbitmq-1      Up 37 minutes (healthy)
rent-lab5-messaging-db-1  Up 37 minutes (healthy)
rent-lab5-deals-db-1      Up 37 minutes (healthy)
rent-lab5-catalog-db-1    Up 37 minutes (healthy)
rent-lab5-auth-db-1       Up 37 minutes (healthy)
```

Рисунок 1 — Первичная проверка ресурсов и контейнеров приложения

2 Подготовка файлов мониторинга

Создал отдельную ветку lab6 и папку labs/lab6/monitoring. Проверил сеть rent-lab5 и драйвер журналов json-file: эта сеть нужна для HTTP-проверок сервисов, а логи контейнеров будут собираться в Loki.

```
[vboxuser@foransible:~$ sudo docker network ls
NETWORK ID     NAME        DRIVER    SCOPE
f93d4c7a7268   bridge      bridge    local
178294bcda7a   host        host      local
0f69cd43df31   none        null      local
085c7c52cb30   rent-lab5   bridge    local
[vboxuser@foransible:~$ sudo docker info --format '{{.LoggingDriver}}'
json-file
vboxuser@foransible:~$
```

Рисунок 2 — Сеть приложения и драйвер журналов Docker

3 Запуск Prometheus и Grafana

Подготовил compose.yml и prometheus.yml, затем скопировал их в ~/lab6-monitoring на Debian. Запустил Prometheus, Grafana и Node Exporter. Для Prometheus установил опрос раз в 15 секунд и хранение метрик за три дня.

```
[+] up 32/32
 ✔ Image grafana/grafana:11.2.2              Pulled                    109.3s
 ✔ Image prom/prometheus:v2.54.1             Pulled                     92.8s
 ✔ Image prom/node-exporter:v1.8.2           Pulled                     18.2s
 ✔ Network lab6-monitoring_monitoring        Created                     0.1s
 ✔ Volume lab6-monitoring_grafana-data       Created                     0.0s
 ✔ Volume lab6-monitoring_prometheus-data    Created                     0.0s
 ✔ Container lab6-monitoring-node-exporter-1 Started                     2.1s
 ✔ Container lab6-monitoring-prometheus-1    Started                     2.0s
 ✔ Container lab6-monitoring-grafana-1       Started                     2.2s
[vboxuser@foransible:~/lab6-monitoring$ sudo docker compose ps
NAME                              IMAGE                          COMMAND
       SERVICE         CREATED          STATUS          PORTS
lab6-monitoring-grafana-1         grafana/grafana:11.2.2         "/run.sh"
       grafana         46 seconds ago   Up 44 seconds   127.0.0.1:13001->3000/tc
p
lab6-monitoring-node-exporter-1   prom/node-exporter:v1.8.2      "/bin/node_exporte
r …"   node-exporter   46 seconds ago   Up 44 seconds
lab6-monitoring-prometheus-1      prom/prometheus:v2.54.1        "/bin/prometheus -
-c…"   prometheus      46 seconds ago   Up 44 seconds   9090/tcp
[vboxuser@foransible:~/lab6-monitoring$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda3       8.6G  3.9G  4.3G  48% /
```

Рисунок 3 — Запуск контейнеров базового мониторинга

4 Подключение источника метрик

Открыл Grafana через SSH-туннель и добавил Prometheus как источник данных с адресом http://prometheus:9090. В Explore выполнил запрос up и получил значение 1 для Prometheus и Node Exporter — сбор метрик работает.

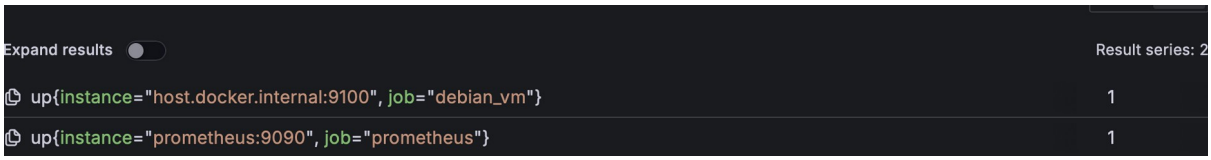


Рисунок 4 — Успешный сбор метрик Prometheus и Debian

5 Графики CPU и RAM

Создал дашборд «Мониторинг микросервисов» и добавил графики загрузки CPU и использования RAM. Значения вывел в процентах, чтобы видеть, какая часть ресурсов виртуальной машины занята.

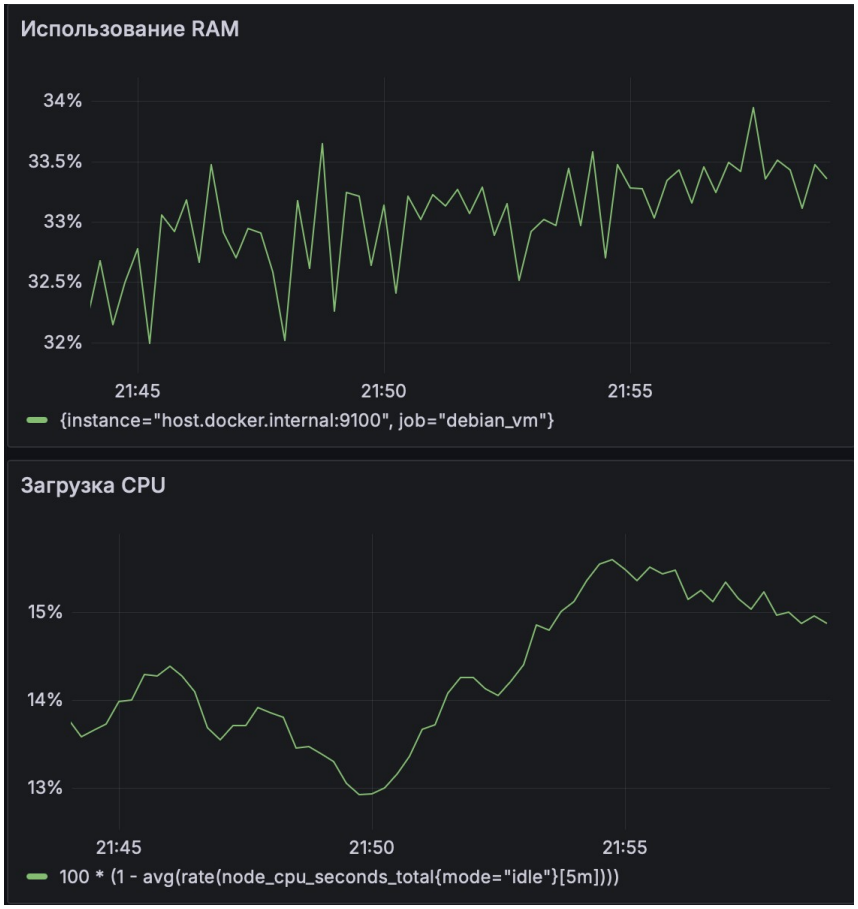


Рисунок 5 — Использование оперативной памяти и загрузка CPU

6 Графики диска и сети

Добавил использование корневой файловой системы и входящий и исходящий сетевой трафик. Для диска выбрал проценты, для сети — байты в секунду. Так можно следить за свободным местом и обменом данными.

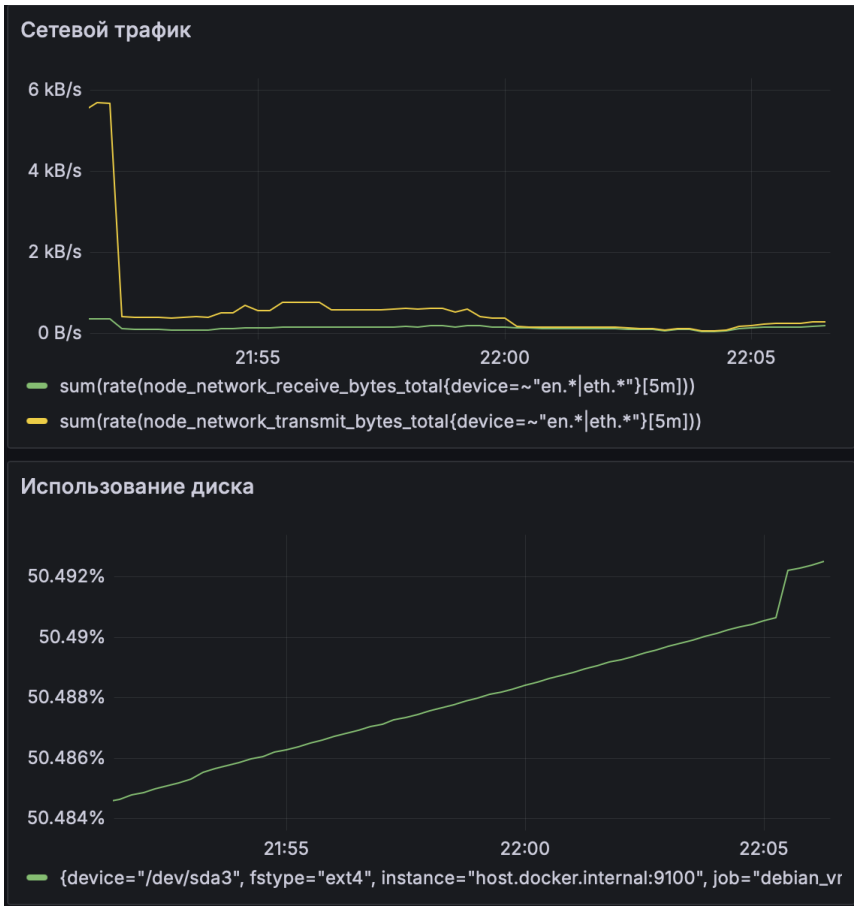


Рисунок 6 — Сетевой трафик и использование диска Debian

7 HTTP проверки микросервисов

Добавил Blackbox Exporter и подключил его к сети rent-lab5. Для gateway и catalog настроил ожидаемый ответ HTTP 200, а для защищённых маршрутов auth, deals и messaging — HTTP 401 без токена. В Prometheus все пять проверок probe_success вернули 1.



Рисунок 7 — Пять успешных HTTP проверок микросервисов

8 Панель доступности

Создал панель Stat для пяти микросервисов и вывел их названия вместо длинных адресов. Настроил отображение последнего состояния: 1 — «Доступен», 0 — «Недоступен».



Рисунок 8 — Состояние gateway auth catalog deals и messaging

9 Сбор и просмотр логов

Добавил Loki и Promtail через compose.logs.yml и настроил чтение журналов через Docker API. В Grafana подключил Loki и вывел общие логи контейнеров на дашборд. Отдельно проверил фильтр по имени gateway, чтобы быстро находить записи нужного сервиса.

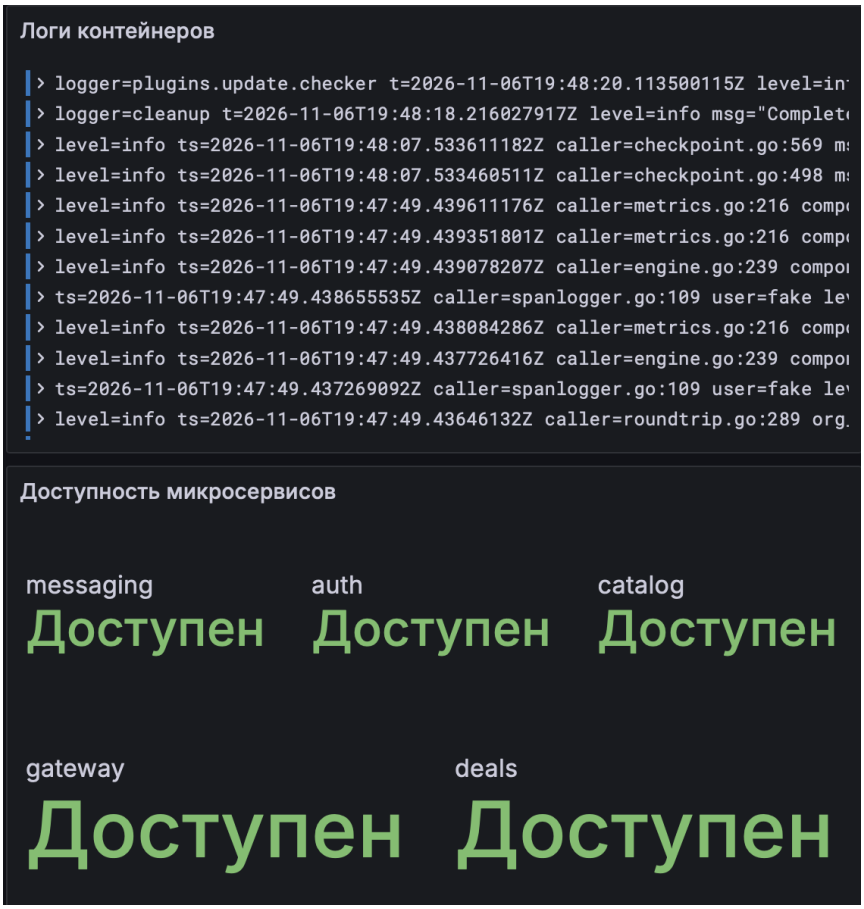


Рисунок 9 — Логи контейнеров и доступность на одном дашборде

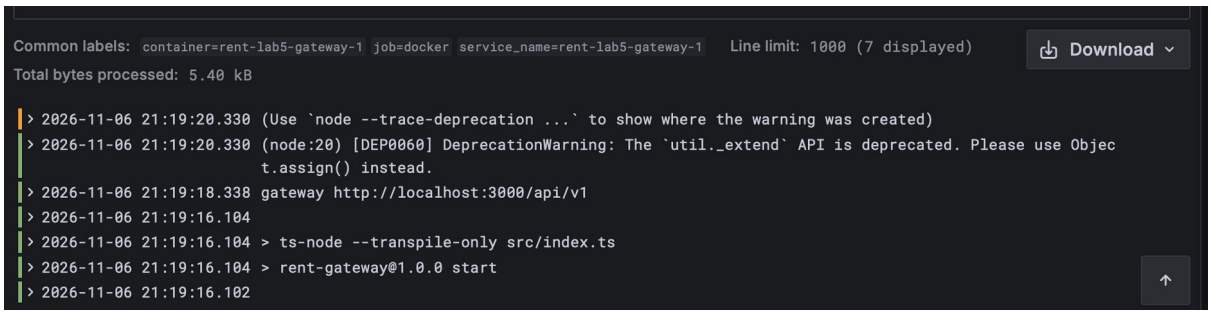


Рисунок 10 — Просмотр логов отдельного контейнера gateway

10 Почтовые уведомления Grafana

Создал пароль приложения Gmail и настроил SMTP через отдельный файл .grafana-smtp.env. Подключил его через compose.smtp.yml и пересоздал Grafana. Пароль сохранил на Debian, отдельно от репозитория.

```
printf '%s\n' \
  "GF_SMTP_ENABLED=true" \
  "GF_SMTP_HOST=smtp.gmail.com:587" \
  "GF_SMTP_USER=$grafana_mail" \
  "GF_SMTP_PASSWORD=$grafana_app_password" \
  "GF_SMTP_FROM_ADDRESS=$grafana_mail" \
  "GF_SMTP_FROM_NAME=Grafana" \
  > .grafana-smtp.env

unset grafana_mail grafana_app_password
Адрес Gmail: tefxy2384@gmail.com
Пароль приложения Google:
vboxuser@foransible:~/lab6-monitoring$ sudo docker compose -f compose.yml -f com
pose.health.yml -f compose.logs.yml -f compose.smtp.yml up -d grafana
[sudo] password for vboxuser:
[+] up 1/1
 ✔ Container lab6-monitoring-grafana-1 Started                              0.8s
vboxuser@foransible:~/lab6-monitoring$ sudo docker compose -f compose.yml -f com
pose.health.yml -f compose.logs.yml -f compose.smtp.yml ps grafana
NAME                        IMAGE                    COMMAND     SERVICE   CREAT
ED          STATUS          PORTS
lab6-monitoring-grafana-1   grafana/grafana:11.2.2   "/run.sh"   grafana   17 se
conds ago   Up 16 seconds   127.0.0.1:13001->3000/tcp
```

Рисунок 11 — Подключение SMTP и запуск Grafana с почтовыми настройками

11 Проверка почты и создание правила

В Contact points добавил Email-контакт и получил тестовое письмо. Создал правило «Микросервис недоступен» для probe_success ниже 1, с проверкой каждую минуту и ожиданием 1m. Пока все сервисы доступны, правило находится в состоянии Normal.

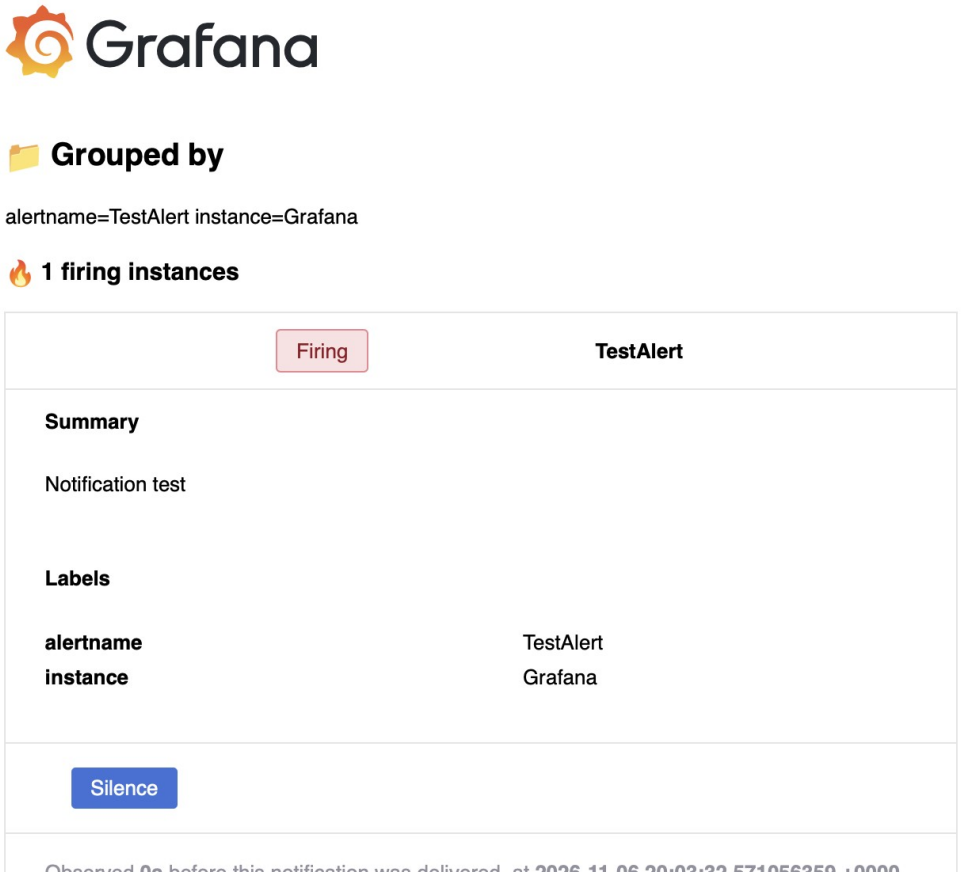


Рисунок 12 — Тестовое письмо от Grafana

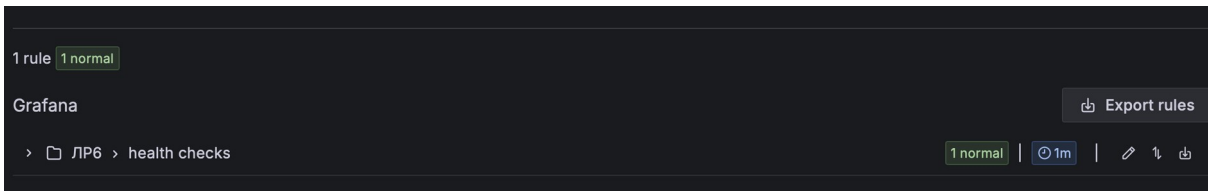


Рисунок 13 — Правило уведомления в группе health checks

12 Проверка уведомления при сбое

Остановил контейнер messaging: на дашборде сервис стал недоступен, а правило перешло через Pending в Firing. На Gmail пришло письмо с названием правила и адресом недоступного сервиса. После проверки запустил контейнер обратно.

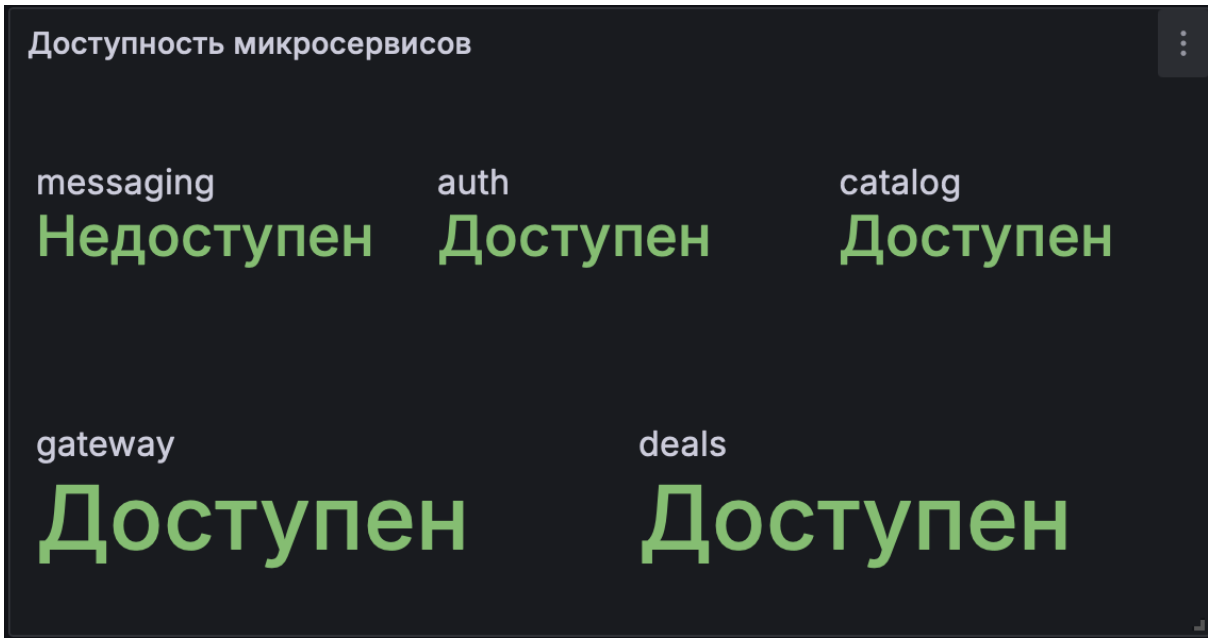


Рисунок 14 — Недоступность messaging после остановки контейнера

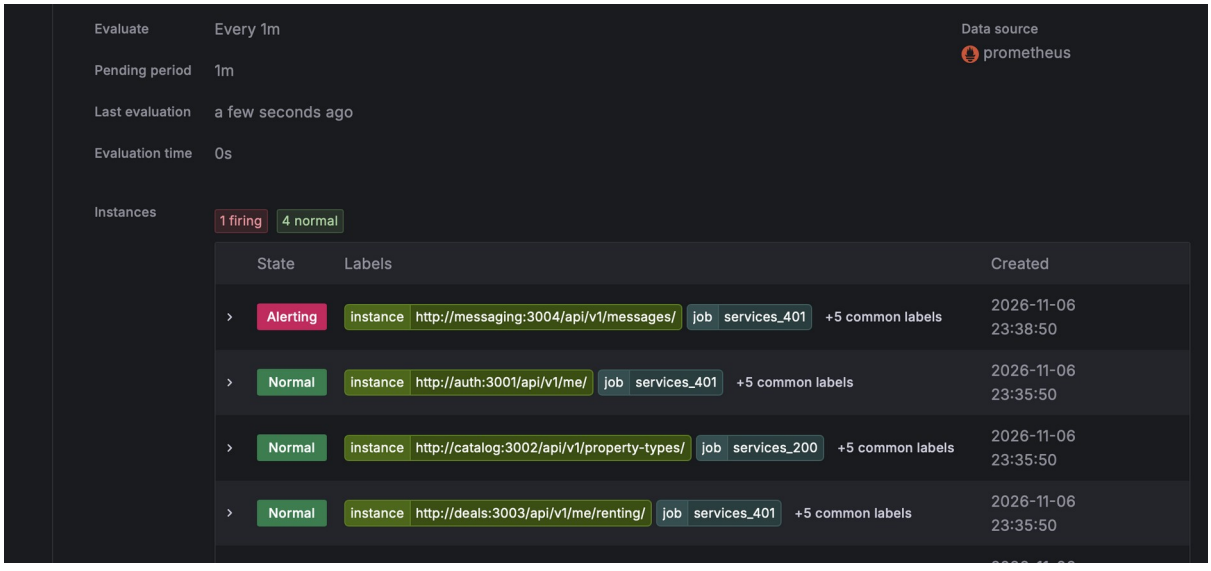


Рисунок 15 — Сработавшее правило для messaging

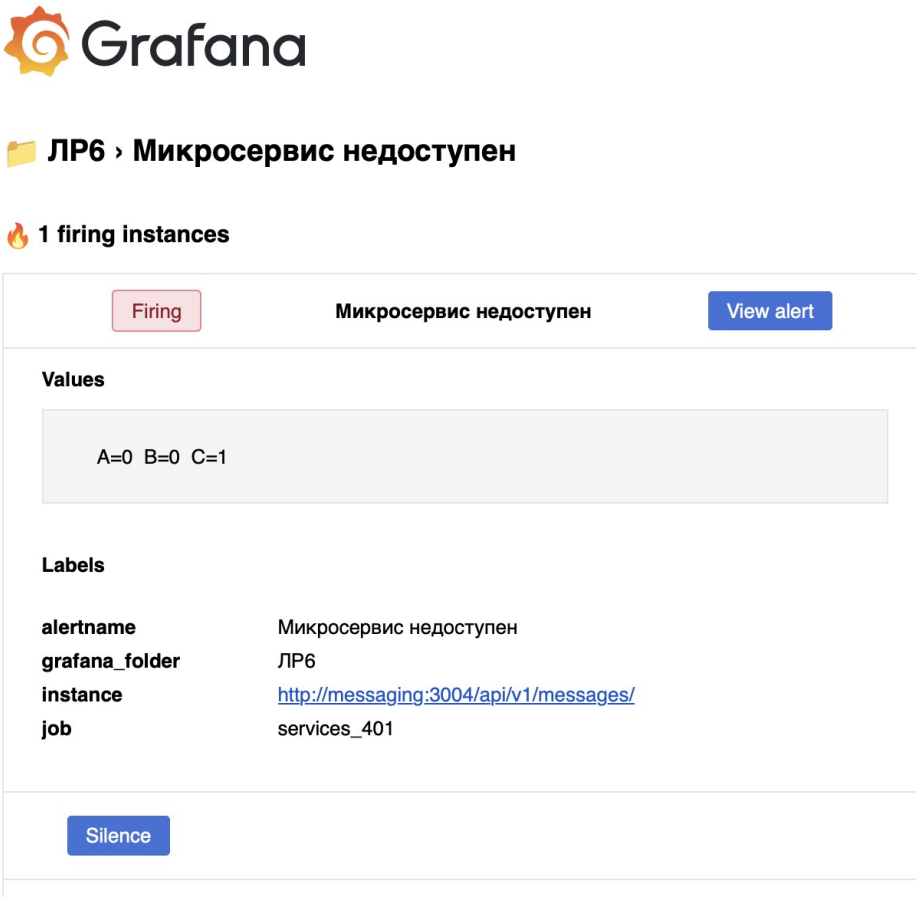


Рисунок 16 — Почтовое уведомление о реальном сбое сервиса

13 Развёртывание своего Sentry

Для Sentry создал отдельную ВМ в Yandex Cloud с Ubuntu 24.04, 4 vCPU, 8 ГБ RAM и диском около 50 ГБ. Установил Docker и Compose, скачал self-hosted Sentry 26.10.0 и запустил профиль errors-only. Веб-интерфейс открыл через SSH-туннель к порту 9000 сервера.

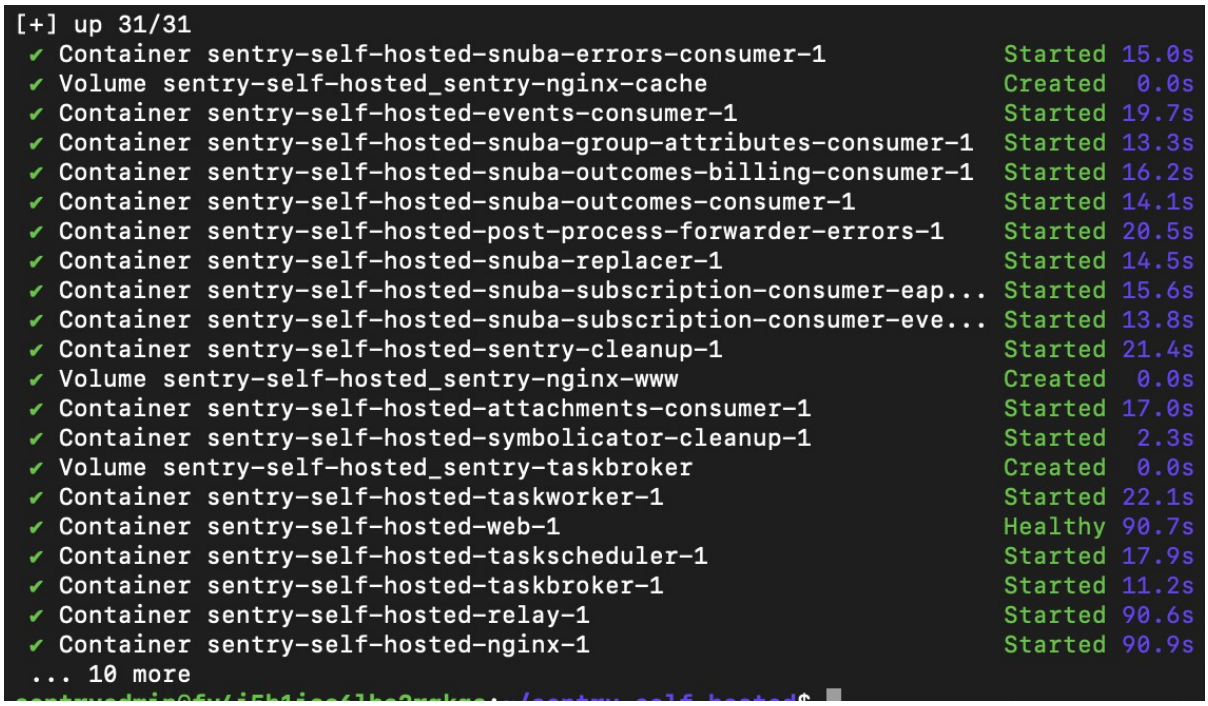


Рисунок 17 — Запуск контейнеров self-hosted Sentry

14 Первичная настройка Sentry

Создал администратора и вошёл в свой Sentry через http://127.0.0.1:19000. Для входа через этот адрес добавил его в CSRF_TRUSTED_ORIGINS. В мастере указал Root URL и почту администратора.

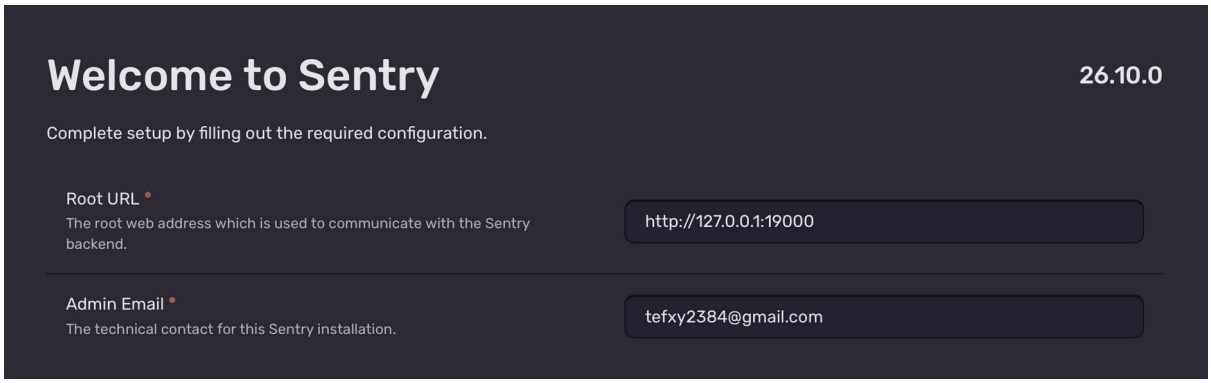


Рисунок 18 — Первичная настройка self-hosted Sentry

15 Создание проекта и подключение SDK

В Sentry создал Express-проект rent-backend и установил @sentry/node в gateway. В index.ts добавил инициализацию из SENTRY_DSN и обработчик с Sentry.captureException. Для проверки добавил маршрут /__sentry-test, включаемый отдельной переменной окружения.

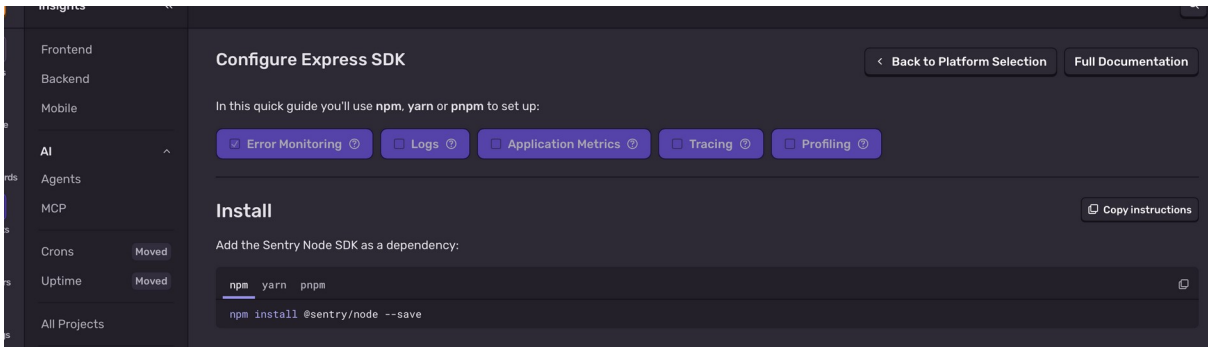


Рисунок 19 — Проект rent-backend и инструкция подключения SDK

```
14  import * as Sentry from "@sentry/node";
15
16  dotenv.config({ path: path.join(__dirname, "../../../.env") });
17
18  if (process.env.SENTRY_DSN) {
19    Sentry.init({
20      dsn: process.env.SENTRY_DSN,
21      environment: "lab6",
22    });
23  }
```

Рисунок 20 — Инициализация Sentry в коде gateway

16 Соединение приложения с Sentry

На Debian открыл отдельный SSH-туннель с адреса Docker-шлюза на сервер Sentry. В DSN использовал этот адрес и порт 19001, чтобы gateway мог обратиться к Sentry из контейнера. Проверочный запрос через туннель получил HTTP 302 на страницу входа.

```
[vboxuser@foransible:~$ sentry_bridge_ip=$(sudo docker network inspect rent-lab5 ]
 --format '{{range .IPAM.Config}}{{.Gateway}}{{end}}')
[vboxuser@foransible:~$ echo "$sentry_bridge_ip"                               ]
 172.18.0.1
```

Рисунок 21 — Адрес шлюза сети rent-lab5

```
vboxuser@foransible:~$ curl -I --max-time 10 "http://${sentry_bridge_ip}:19001/"
HTTP/1.1 302 Found
Server: nginx
Date: Fri, 06 Nov 2026 23:41:24 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
location: /auth/login/
x-robots-tag: noindex, nofollow
vary: Accept-Language, Cookie
content-language: en
x-frame-options: deny
x-content-type-options: nosniff
```

Рисунок 22 — Ответ Sentry через туннель из Debian

17 Обновление gateway и тест ошибки

Из ветки lab6 собрал образ rent-lab6-gateway:1 и подключил его через compose.sentry.yml. Перезапустил только gateway с файлом sentry.env, затем вызвал /__sentry-test. Маршрут специально вернул HTTP 500, чтобы проверить отправку исключения.

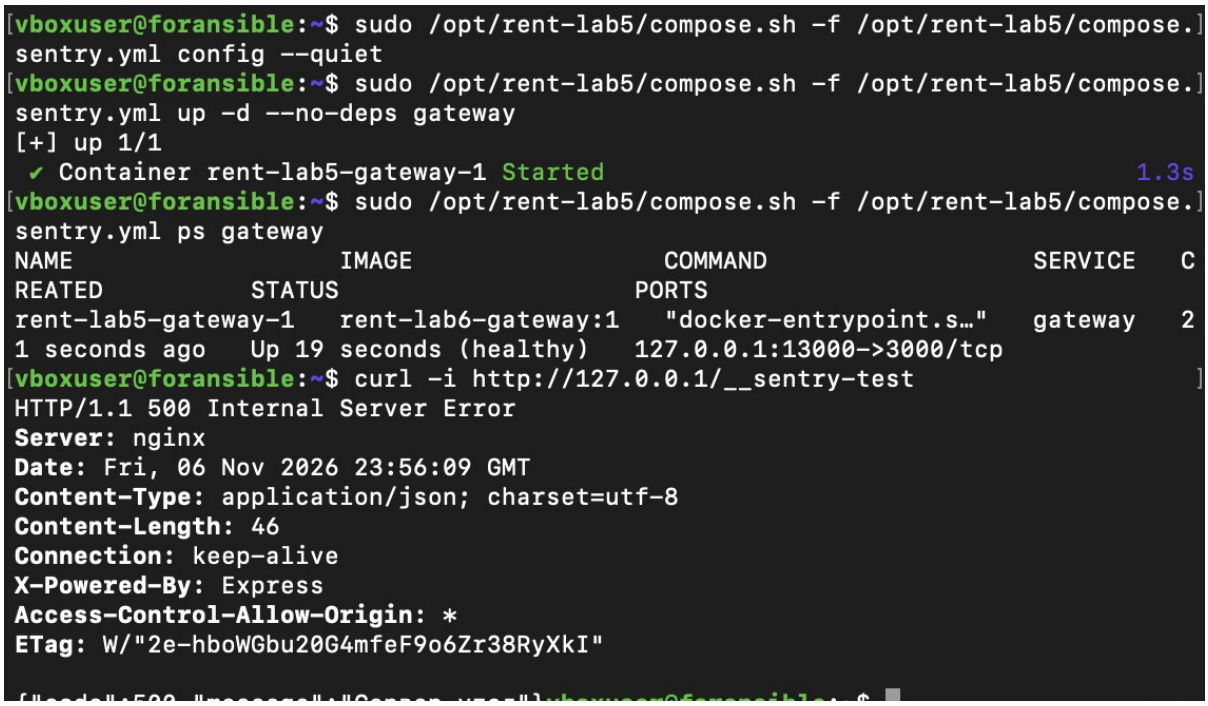


Рисунок 23 — Обновлённый gateway и вызов тестовой ошибки

18 Проверка события и завершение работы

В проекте rent-backend появилось событие «Тестовая ошибка лабораторной работы» с маршрутом GET /__sentry-test и средой lab6. После проверки установил SENTRY_TEST_ENABLED=0 и пересоздал gateway, чтобы выключить тестовый маршрут. Конфигурации мониторинга, dashboard.json и изменения SDK сохранил в ветке lab6.

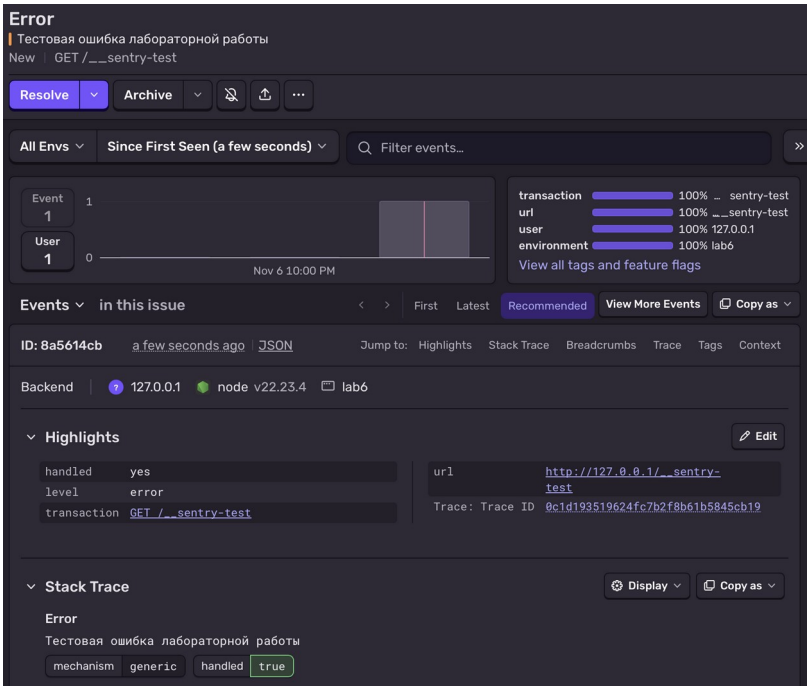


Рисунок 24 — Тестовая ошибка приложения в self-hosted Sentry

```
[{"code":500,"message":"Сервер упал"}vboxuser@foransible:~$ sudo sed -i 's/^SENTR]
Y_TEST_ENABLED=1$/SENTRY_TEST_ENABLED=0/' /etc/rent-lab5/sentry.env
[vboxuser@foransible:~$ sudo /opt/rent-lab5/compose.sh -f /opt/rent-lab5/compose.]
sentry.yml up -d --no-deps gateway
[+] up 1/1
 ✔ Container rent-lab5-gateway-1 Started                                    0.8s
[vboxuser@foransible:~$ sudo /opt/rent-lab5/compose.sh -f /opt/rent-lab5/compose.]
sentry.yml ps gateway
NAME                  IMAGE                  COMMAND                  SERVICE   C
REATED         STATUS                              PORTS
rent-lab5-gateway-1   rent-lab6-gateway:1   "docker-entrypoint.s…"   gateway   5
 seconds ago   Up 4 seconds (health: starting)   127.0.0.1:13000->3000/tcp
```

Рисунок 25 — Отключение тестового маршрута после проверки

Вывод

Настроил дашборд с ресурсами Debian, доступностью пяти микросервисов и логами контейнеров. Проверил уведомление Grafana на Gmail при остановке messaging. Развернул свой Sentry на отдельной ВМ и получил в нём тестовую ошибку gateway через SDK.